

Event Management System: Requirements, Architecture, Design and Validation

Saketh Narayanam — PES1UG24CS292 | Neeraj R Gowda — PES1UG24CS295 | Likhith Reddy M — PES1UG24CS282 | N. Adil Ahmed — PES1UG24CS285

Team E7

Abstract—This paper presents a web-based Event Management System designed to centralize event discovery, registration, participant management, organizer operations, notifications, reporting, and administration. The system addresses manual registration, duplicated participant records, capacity tracking, inconsistent event updates, and administrative effort. The proposed system uses a layered client-server architecture with React.js at the presentation layer, Node.js/Express.js for application and API logic, and a database layer for persistent user, event, registration, and reporting data. The design defines four principal actors: Visitor, User/Attendee, Organizer, and Administrator. Functional, non-functional, and security requirements are traced to use cases and test cases. Validation covers functionality, security, performance, usability, reliability, and maintainability.

Keywords—Event Management System, React.js, Node.js, Express.js, Event Registration, Capacity Management, Role-Based Authorization, UML, Software Testing

I. INTRODUCTION

Event information and registration activities can become difficult to manage when records are maintained manually or across multiple systems. The project documents identify problems including slow and repetitive manual registration, duplicated or incomplete participant records, manual capacity tracking, inconsistent communication of event changes, and additional effort required for reporting and administration. The proposed Event Management System brings these activities into one web-based platform.

II. PROBLEM AND OBJECTIVES

The system is intended to make event discovery and registration easier while providing organizers with tools for event and participant management. Its objectives include providing one place for event information and registrations, enabling online registration and cancellation, preventing registration beyond event capacity, supporting participant management and event updates, controlling administrative access, maintaining accurate information, and reducing repetitive event-management work.

III. SYSTEM SCOPE AND REQUIREMENTS

The first version covers user registration, login and profile management; event creation, modification, cancellation, search and filtering; event registration and cancellation; participant and capacity management; notifications and event updates; administrator management; and registration/event reports. Physical venue management, catering, transportation, advanced financial accounting, live streaming, and AI-based event recommendations are outside the stated scope.

IV. USE CASE MODEL

The use-case model defines four main actors. A Visitor is an unregistered person who can browse, search/filter, view events, and register an account. A User/Attendee can log in, register or cancel registrations, view registered events, update a profile, and receive notifications. An Organizer can create, edit and cancel events, set capacity, view participants / registration counts, send updates, and generate registration reports. An Administrator manages users, organizers, events, and system-level reports.

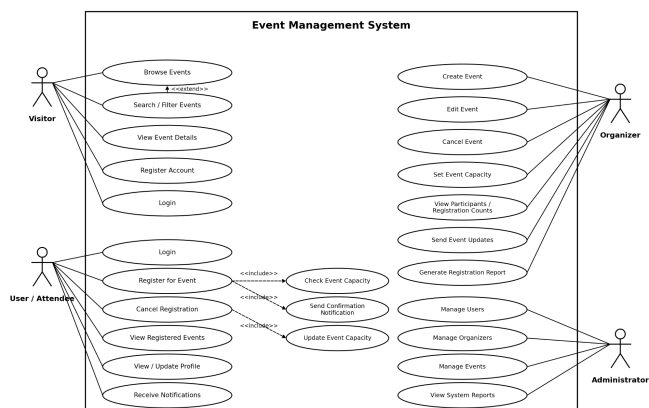


Fig. 1. UML use-case diagram of the Event Management System.

V. SYSTEM ARCHITECTURE

The architecture follows a layered client-server approach. The Presentation Layer uses React components, pages, forms, navigation, validation messages, and responsive UI. The Application/API Layer provides Express.js routes, authentication, authorization, business rules, registration logic, capacity checks, and notification handling. The Data Access Layer manages database queries, models, validation, and persistence. The Database Layer stores accounts, roles, events, registrations, and report-related information.

Event Management System - Component View

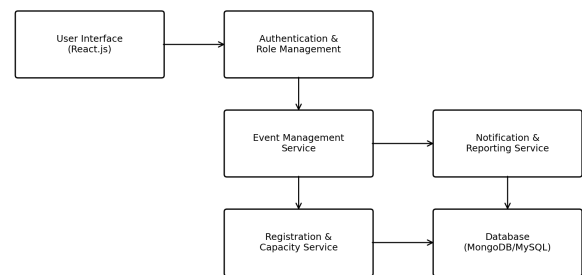


Fig. 2. Component view of the Event Management System.

VI. DETAILED DESIGN AND API

The detailed design focuses on interactions among the user interface, API layer, and database. The login flow verifies credentials through the authentication API and database before returning a token or session. The event-registration flow authenticates the user, checks event existence and availability, compares registration count with capacity, prevents duplicate registration, creates the registration only after validation succeeds, and returns confirmation. The API design includes endpoints for authentication, event discovery and details, registration and cancellation, organizer event management, participant information, event updates, administration, and registration reports. Common responses include 200, 201, 400, 401, 403, 404, 409, 422, and 500.

VII. SECURITY

Security requirements include secure one-way password hashing, authentication and role-based authorization for protected functions, input validation, secure session handling, and restriction of administrative actions to authorized administrator accounts. The architecture places authentication at the API boundary and checks authorization before protected organizer or administrator operations. Database access is isolated behind the application/data-access layer rather than being exposed directly to the browser.

VIII. TESTING AND VALIDATION

Testing covers account creation and authentication, event browsing and search, registration and cancellation, capacity validation, duplicate-registration prevention, organizer and administrator functions, notifications, reporting, performance, usability, reliability, and security. The test cases include boundary checks for full events, duplicate registration prevention, unauthorized administrator access, password protection, responsive behavior, registration consistency, malicious input and expired sessions, organizer reports, and administrative actions. The stated performance targets are event listing within 3 seconds and valid event registration within 2 seconds under normal conditions.

IX. REQUIREMENTS TRACEABILITY

The project maintains traceability from requirements to use cases and test cases. Functional requirements FR-01 through FR-11 cover authentication, event discovery, details, registration, capacity, organizer management, updates, reporting, and administration. Non-functional requirements NFR-01 through NFR-07 cover performance, security, usability, reliability, and maintainability. Security requirements SEC-01 through SEC-05 define password protection, authorization, input validation, session security, and administrator restrictions. The RTM maps these requirements to UC-01 through UC-20 and the corresponding test cases.

X. CONCLUSION

The Event Management System provides a structured web-based approach for event discovery, registration, capacity and participant management, organizer operations, notifications, reporting, and administration. The documented architecture, use cases, API behavior, security controls, test cases, and traceability matrix provide a consistent basis for implementation and validation. The project documentation also clearly identifies the first-version scope and the functions reserved for future work.

REFERENCES

- [1] ISO/IEC/IEEE 29148:2018, Systems and software engineering — Life cycle processes — Requirements engineering.
- [2] Event Management System Project Documents, Team E7: Problem Statement, Feasibility Analysis, Project Scope, SRS, Security Objectives

and Requirements, Use Case Model, Software Test Plan, Software Architecture, Detailed Design, API and Error Handling, Test Cases, RTM, and Validation and Testing Summary.